

Programmmentwurf 2

Vorwort

Der Programmmentwurf 2 der Vorlesung Mikrocomputertechnik legt den Schwerpunkt auf

- Power-Modes (Wakeup, Shutdown)
- Nicht-Flüchtiger Speicher (Emuliertes EEPROM)
- Serielle Schnittstellen (UART, I2C)

Hinweis: Betätigen bezeichnet die Zustandsänderung einer Taste von ungedrückt zu gedrückt.

Hardware-Anforderung

HW1: Als Plattform soll ein Nucleo Board mit STM32G474RE und EduShield verwendet werden.

HW2: Als Beschleunigungssensor soll ein GY-521 Sensorkit verwendet werden.

HW3: Es sind keine weiteren elektronischen Schaltungen für den Programmmentwurf zulässig.

Allgemeine Software Anforderung

SW1: Ausgangsbasis für den Programmmentwurf soll das anhängte ZIP-Archiv verwendet werden.

SW2: Der zyklische Teil der Umsetzung soll im Kontext der Funktion RunState_Cyclic_Running aufgerufen werden. Erlaubt sind die Übergabe des mainState Counters sowie Aufrufe von öffentlichen Funktionen.

SW3: Die Initialisierung der Hardware soll im Kontext der main-Funktion vor Aufruf der RunState_Init() erfolgen.

SW4: Für die logische Funktionsweise erforderliche Initialisierungen der Umsetzungen dürfen im Kontext der RunState_Init() aufgerufen werden.

Sleep Wakeup Verhalten

SW5: Durch Betätigen des User Buttons B1 auf dem Nucleo Board soll der Prozessor geweckt werden.

SW6: Der Prozessor soll vom RUN Mode in den Shutdown Mode überführt werden, wenn der Taster ein zweites Mal gelöst wurde.

Hinweis: Erst wenn im RUN Mode der B1 gelöst war und erneut gedrückt wird, soll beim Lösen des Tasters der Shutdown Mode erreicht werden.

SW7: Solange der Prozessor im RUN Mode verweilt, soll die LED D1 auf dem EduShield mit einer Frequenz von 1 Hz blinken.

SW8: Sofern nicht durch Anforderung SW5 geweckt, soll der Prozessor nach 10 Sekunden aufwachen.

SW9: Um ein definierter Spannungspegel zu gewährleisten, sollen am Wakeup-Pin geeignete Maßnahmen getroffen werden.

SW10: Wird der Prozessor in Folge von SW5 geweckt, soll die LED D2 auf dem EduShield leuchten.

SW11: Wird der Prozessor in Folge von SW8 geweckt, soll die LED D3 auf dem EduShield leuchten.

SW12: Die Anforderungen SW5 – SW11 sollen in der Datei powermodes.c umgesetzt werden.

Datenspeicher

SW13: Für Umsetzung von SW14 – SW18 soll der STM eigene eeprom-Emulator verwendet werden.

SW14: Der Eeprom-Emulator soll so konfiguriert werden, dass die ersten zwei Pages der zweiten Flash-Bank als Datenspeicher verwendet werden.

SW15: Liegt zum Aufrufzeitpunkt der MainState_Init() keine gültige Partition vor, soll der Flash entsprechend der eeprom-Emulation partitioniert werden.

SW16: Um die Anzahl der Startup-Sequenzen nach der Partitionierung in SW15 zu speichern, soll eine Variable mit 32 Bit Auflösung und Identifier 1 verwendet werden. Eine neue Anzahl soll zum frühestmöglichen Zeitpunkt persistiert werden.

SW17: Um die akkumulierte Laufzeit des Prozessors in Millisekunden nach der Partitionierung in SW15 zu speichern, soll eine Variable mit 32 Bit Auflösung und Identifier 2 verwendet werden.

SW18: Die akkumulierte Laufzeit soll bei Bedingung SW6 und zusätzlich alle 5 Sekunden persistiert werden.

SW19: Die Anforderungen SW14 – SW18 sollen in der Datei storage.c umgesetzt werden.

Beschleunigungssensor

SW20: Der Prozessor soll den Beschleunigungssensor GY-521 über die I2C-Schnittstelle an den Pins PC8 und PC9 auslesen.

SW21: Ab dem Aufrufzeitpunkt von MainState_Init() soll der GY-521 über den Pin PC5 mit Spannung versorgt werden.

SW22: Wird im RUN State der Taster SW2 auf dem Edushield betätigt, soll einmalig der GY-521 konfiguriert werden.

SW23: Nachdem die Konfiguration abgeschlossen ist, soll die LED D0 auf dem EduShield leuchten.

SW24: Solange die LED D0 leuchtet, sollen die Inertialdaten den GY-521 sekundlich ausgelesen, aufbereitet und ausgegeben werden.

SW25: Der GY-521 soll folgendermaßen konfiguriert werden:

- Drehrate: +-1000 °/s
- Linearbeschleunigung: +- 16g
- Tiefpassfilter: 10 Hz aktiv

SW26: Vor Erreichen des Shutdown Modes soll die Spannungsversorgung des GY-521 explizit deaktiviert werden und alle LEDs aus sein.

SW27: Sowohl für das Konfigurieren als auch für das zyklische Auslesen soll eine DMA Lösung implementiert werden.

SW28: Die Anforderungen SW20 – SW27 sollen in der Datei acceleration.c umgesetzt werden.

Datenaufbereitung

SW29: Die ausgelesenen Rohwerte der Beschleunigungsdaten sollen folgendermaßen in das UTF8-Format konvertiert werden: „Drehrate: VAAA VBBB VCCC; Linear: VXX.X, VYY.Y, VZZ.Z“.

SW30: Die absoluten Maximalwerte der Linearbeschleunigung sollen auf 99.9 beschränkt sein.

SW31: Mit den Majukeln X, Y, Z werden die Linearbeschleunigungen in x-, y-, z-Richtung bezeichnet.

SW32: Mit den Majukeln A, B, C werden die Drehraten um die x-, y-, z-Achse bezeichnet.

SW33: Mit V wird das Vorzeichen der Werte bezeichnet. Das Zeichen ‚+‘ soll für Werte ≥ 0 verwendet werden, das Zeichen ‚-‘ für alle anderen Werte.

SW34: Die Linearbeschleunigungen sollen in SI-Einheiten ausgegeben werden.

SW35: Die Drehraten sollen in Grad pro Sekunde ausgegeben werden.

SW36: Die Rotationsgeschwindigkeiten sollen in Grad pro Sekunde ausgegeben werden.

Ausgabe über UART

SW37: Die aufbereiteten Daten aus SW28 – SW36 sollen über die LPUART-Schnittstelle des Prozessors am PA2, PA3 ausgegeben werden.

SW38: Der LPUART ist zu konfigurieren mit 115200 Bits/s, Data Size = 8, Parity = 0, Stop Bits = 1.

SW39: Das Versenden der aufbereiteten Daten soll durch eine DMA Lösung umgesetzt werden.

SW40: Die Anforderungen SW29 – SW39 sollen in der Datei processing.c umgesetzt werden

Qualitäts-Anforderung

Q1: Der Programmentwurf soll in der Sprache C (C99) abgefasst werden.

Q2: Der Quellcode soll für eine ARM GNU Toolchain ab Version 12.3.Rel1 kompilier- und linkfähig sein.

Q3: Die C- und H- Dateien im Projektordner /source sollen mit Astyle formatiert werden.

Q4: Die C-Dateien sollen dem Symbol C_SOURCES in der FileList.mak hinzugefügt werden. C_SOURCE ist eine Liste, wobei jede Zeile zwischen zwei Pfaden mit einem Backslash \ beendet wird.

Q5: Der Programmfluss innerhalb der Algorithmen sowie der Prototyp der Funktion sollen in Englischer Sprache nachvollziehbar dokumentiert werden.

Q6: Doxygen darf keine Warnungen erzeugen.

Q7: In Software darf nicht mehr umgesetzt sein als durch die Anforderungen des Programmentwurfs gefordert ist.

Q8: Das Programm soll ablauffähig auf dem Nucleo Board im Auslieferungszustand sein.

Q9: Der Programmentwurf ist eigenständig zu konzeptionieren, zu programmieren, zu bauen und gemäß Lieferanforderungen gebündelt abzugeben. Insbesondere ist die Verwendung von generativer künstlicher Intelligenz (z.B. ChatGPT) nicht zulässig. Darüber hinaus darf Quelltext nicht zwischen Teilnehmern der Prüfung geteilt werden. Davon abweichend dürfen alle Aufgaben im Kontext der Vorlesung benutzt werden.

Q10: Peripherie soll mit CubeMX konfiguriert werden.

Q11: Der generierte Code aus CubeMX soll zur Konfiguration der Peripherie in der main()-Funktion vor der MainState_Init() aufgerufen werden.

Lieferumfang und Abgabetermin

L1: Die Abgabe des Programmentwurfs soll in Form eines ZIP-Archivs erfolgen.

L2: Das Archiv aus L1 soll den Quellcode und die Projektverwaltung enthalten.

L3: Das Archiv aus L1 soll zusätzlich die Ausgabe des Build-Prozesses enthalten.

L4: Der Programmentwurf soll spätestens zum Datum der Abgabe vollständig abgegeben werden. Temporäre Abgaben zählen als Abgaben sofern keine finale Abgabe erfolgt ist. Die Möglichkeit einer Nachabgabe sofern final abgegeben wurde, liegt im Ermessen des Dozenten.

L5: Die Dateien aus L3 sollen funktional reproduzierbar sein durch den Inhalt aus L2.